path_finder

A live monitor for Claude Code sessions that talk to each other. Guide to the code, and how to install it on another computer. Written 2026-10-21.

1. What it does

When several Claude Code sessions work on one project, they send each other messages over local sockets. Nothing shows you that traffic. path_finder reads each session's transcript file, works out who sent what to whom, and draws it live in a small window: one pod per session, arrows for messages in flight, a tether for a session waiting on another, a line saying what each session is doing, and a side panel listing the project's tasks.

Everything runs on one machine and reads files only. No hooks, no central server, nothing is written to Claude Code's own files.

2. The pieces

Path	What it is
`discovery/discover.py`	Session discovery. Each session registers its role once; the script lists live sessions from Claude Code's registry plus those identity files.
`viz/watcher.py`	The backend. Tails every transcript, keeps the state in memory, serves the pages and two endpoints on port 4780, writes an event log.
`viz/web.html`	The compact window, about 380 px wide. This is the page you use.
`viz/index.html`	The full-width page with cards and connectors. Same data, larger layout.
`path_finder/`, `pyproject.toml`	The installable command `path-finder`. Thin wrappers that run the two scripts above in place, plus the `window` command.
`plugin/`, `.claude-plugin/marketplace.json`	The Claude Code plugin with two skills, and the marketplace manifest that lets you install it from the checkout.
`tests/`	405 pytest tests: unit, integration, sanitized transcript fixtures, and a Node harness that renders the page.
`docs/`	The transcript format, the message format, the visualization plan, and the reference UI frames.
`SPEC.md`, `CLAUDE.md`, `.claude/roles/`	The task list, the project rules every session reads first, and one file per team role.

3. How the data flows

```
~/.claude/sessions/<pid>.json     who is running, name, start folder
discovery/identity/<pid>.json     the role each session registered
~/.claude/projects/.../<id>.jsonl  each session's transcript
        |
        v   polled every 250 ms, complete lines only
  viz/watcher.py --> state in memory --> GET /state   full snapshot
        |                              --> GET /events  stream of changes
        v
  viz/events.jsonl   append-only log, for replay
```

Sessions

A session is keyed by its PID, which names its socket `uds:/tmp/cc-socks/<pid>.sock`. The watcher finds its transcript through the session id in the registry file. Each session carries a status (Working or Idle), an inbox count, an activity line, and a list of sessions it is waiting on.

Messages

A message is keyed by its `msg_id`, which appears in the sender's tool result and again in the receiver's transcript, so the two sides join. Stages:

- **Pending**: the sender called SendMessage, no result yet.
- **In inbox**: the result said success, or the receiver's queue logged the arrival.
- **Read**: the receiver's transcript shows the message delivered to its Claude, either as a normal entry when it was idle or as a queued-command attachment when it was busy.
- **Failed**: the result said not sent, or a later delivery notice said the message was dropped.

Waiting

A session is waiting on another when it asked for an idle notice from it, or when a message it sent is still Pending or In inbox. The wait ends when the idle notice lands, the message is read, or it fails.

Activity

While a session works, its line is the description of its latest tool call, with the tool name stripped. When it is idle the line is blank, unless it is waiting, in which case it reads "waiting on" plus the other session's role.

Tasks

The watcher parses the Tasks section of `SPEC.md` whenever the file changes. The manager marks a task `[~]` when it assigns it and `[x]` when it passes review, so the file is the source of truth for the panel.

4. The window

Top to bottom in `viz/web.html`:

- **The web.** One pod per live session, always shown, in a ring. A pod has a coloured dot, the role, a status pill, an envelope with the number of queued messages when it is above zero, and its activity line. Pods grow to fit their text, and you can drag them anywhere; positions are remembered by role, and a Reset layout button returns the ring.
- **Arrows.** Grey while Pending, green while In inbox, red while Failed. A read arrow stays 30 seconds, muted, so you can follow an exchange, and leaves early when a reply comes back on the same pair. One arrow per direction per pair, with a count when more are queued.
- **Tethers.** A dashed yellow curve with an eye glyph from a waiting session to the one it waits on, for as long as the wait lasts, drawn beside any message arrow on the same pair.
- **Caption.** The transcript entry behind the latest change.
- **Strip.** One chip per session; click to highlight its arrows.
- **Activity rows.** One row per session with its status and activity line. Drag rows into your own order; it is remembered by role.
- **Task panel.** Collapsible, grouped into Discovery, Visualization, and Packaging folders, each closable, with counts. Hover a cut-off title to read it in full; click to see the whole task text. In-progress tasks show their owner and highlight that owner's pod.

5. The command

`path-finder` is installed by pipx or pip from the checkout and runs the scripts in place, so keep the checkout where it is.

Command	Does
`path-finder identify --role R`	Registers this session's role. A blank role is rejected.
`path-finder self`	Prints this session's record; exits 2 until identify has run.
`path-finder list --root DIR`	Lists live sessions under DIR with roles; drops dead PIDs.
`path-finder done --root DIR`	Exits 0 when every live session under DIR has a role.
`path-finder serve [--port 4780]`	Starts the watcher and serves the pages.
`path-finder state`, `map`, `tail`, `replay`	Snapshot, PID to transcript map, raw tail, fixture replay.
`path-finder window [--port 4780] [--page web]`	Starts the watcher if nothing answers on the port, then opens the page in a chromeless 380 by 720 px app window in Chrome, Chromium, Brave, or Edge. Falls back to the default browser with a note.

6. The plugin and its two skills

The plugin is the folder `plugin/`, name `path-finder`. It holds two skills and nothing else, by design: no hooks, and the behaviour stays in the command where it is tested.

/path-finder:identify <role>

1. If no role was given, asks the user one question: what is this session's role.
2. Runs `path-finder identify --role <role>`, or the discovery script from the checkout when the command is not on PATH.
3. Prints the session's own record from `self`.

Run it once per session. Until then the session shows up unidentified, with no role on the page.

/path-finder:mesh

1. Lists the live sessions under the current folder with their roles. No messages are sent, so the other sessions pay nothing.
2. If nothing answers on port 4780, starts the watcher and opens the window.
3. Prints the session lines and the page URL.

7. Installing on another computer

Needs Python 3.9 or newer, pipx (or pip), Claude Code, and Chrome, Brave, or Edge for the popup window. The package has no third-party dependencies.

Step 1. Get the code

The repository is at `https://github.com/kharanga/path_finder`. It must be visible to the person installing: public, or they are added as a collaborator.

```
git clone https://github.com/kharanga/path_finder.git
cd path_finder
```

Step 2. Install the command

```
pipx install .
# or, inside a virtual environment:  pip install -e .
path-finder --help
```

Step 3. Open the window

```
path-finder window
```

This starts the watcher on port 4780 and opens the window. Run it from the project folder you want to watch, or pass `--root DIR`.

Step 4. Install the plugin in Claude Code

For a trial that changes no settings, start a session with the plugin loaded from disk:

```
claude --plugin-dir plugin
```

To keep it, register the checkout as a marketplace (the manifest is in `.claude-plugin/marketplace.json`) and install from it:

```
claude plugin marketplace add /full/path/to/path_finder
claude plugin install path-finder@path_finder
```

Check with `claude plugin list`. To validate the files on their own: `claude plugin validate .` and `claude plugin validate plugin`.

Step 5. Use it

In each Claude Code session on that machine, run `/path-finder:identify <role>` once. Then `/path-finder:mesh` in any session lists the team and opens the window.

> **What the other computer will see.** Only its own sessions. The watcher reads the session registry and transcripts under that user's home folder. There is no shared server, so sessions on two machines never appear in one window.

8. Running the tests

```
python3 -m pytest tests
```

The page tests need Node for the render harness in `tests/page/`. The fixtures under `tests/fixtures/` are sanitized transcript captures with no home paths.

9. Rules worth knowing

- Claude Code's own files (`~/.claude/settings.json`, the sessions registry, the transcripts, the sockets) are read only. Nothing here writes to them.
- Every received message costs the receiver a full turn, so discovery is self-registration: a session registers itself, and everyone else reads the files.
- No hooks. A leftover dashboard hook on the original machine still logs errors in transcripts; it is a user item, not something the code fixes.
- In the team setup, only the manager session commits and pushes, and every task is assigned by ID from `SPEC.md`.